

**MARS ROVER MANIPAL
RESEARCH AI TASKPHASE
Report 3**

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Review questions

1.1 Research questions

1.1.1 What are the methods to evaluate a journal or a conference?

Evaluating a journal

1. Indexing and databases

- Check whether the journal is indexed in Scopus, Web of Science, PubMed, IEEE Xplore, ACM Digital Library etc.
- Predatory journals are usually absent from reputable indexes.

2. Impact metrics

- Impact Factor (IF, Clarivate JCR) show average citations per paper.
- Scimago Journal Rank (SJR) and Source Normalized Impact per Paper (SNIP) (from Scopus).
- h-index of the journal.
- Quartile ranking (Q1–Q4) in Scimago — Q1 being top 25%.

3. Peer-review quality

- Check whether they follow a transparent peer review process.
- Check whether the review time is realistic (weeks–months, not just days).
- Check for multiple expert reviewers, not “instant acceptance.”

4. Publisher reputation

- If it is run by reputable publishers (Elsevier, Springer, IEEE, ACM, Nature, Wiley, Taylor & Francis, etc.).
- Check if it appears on DOAJ (Directory of Open Access Journals).

5. Editorial board

- Whether they are recognized experts or unknown/irrelevant names.
- Whether affiliations are verifiable.

6. Ethics and transparency

- Follows COPE (Committee on Publication Ethics) guidelines.
- Clear policies on plagiarism, conflicts of interest, and retractions.

Evaluating a conference

1. Indexings and proceedings

- Check whether proceedings are published in IEEE Xplore, ACM Digital Library, Springer LNCS, Elsevier, etc.
- If they are indexed in Scopus / Web of Science.

2. Reputation and ranking

- Check *CORE Conference Ranking (A, A, B, C)**.
- Look at the conference's history, like for how many years it has been running.
- Well-known series (NeurIPS, ICML, SIGCOMM, ICCV, AAAI, etc.) carry high weight.

3. Acceptance rate

- Lower acceptance rate (e.g., 15–25%) usually means a higher quality.
- Too high acceptance (70–90%) might mean low selectivity.

4. Keynote speakers and committee

- Whether they invite respected researchers.
- Whether the program is committee credible.

5. Networking and audience

- Check who usually attends. Whether it's a mix of industry and academia.
- Past conference videos/papers can give an insight into the quality.

6. Sponsorship and support

- Whether they are backed by strong societies (IEEE, ACM, IFIP, AAAI, etc.).
- Predatory conferences often lack credible sponsors.

Red flags for both conferences and journals

- Overly fast acceptance promises (“publish in 3 days”).
- Suspiciously broad scope (“Journal of Everything Science”).
- Unverifiable editorial board or organizers.
- Poorly written website with grammar errors.
- Hidden or unclear fees.

1.2 Python questions

1.2.1 What is the function of the `super()` keyword?

In Python, `super()` is mainly used in classes with inheritance to call a method from the parent (superclass) without explicitly naming the parent.

Example

```
class Parent:
    def greet(self):
        print("Hello from Parent")

class Child(Parent):
    def greet(self):
        super().greet() # calls Parent's greet
        print("Hello from Child")

c = Child()
c.greet()
```

Output:

```
Hello from Parent
Hello from Child
```

The `super()` keyword can also be used in constructors.

Example

```
class Parent:
    def __init__(self, name):
        self.name = name
        print(f"Parent initialized with {self.name}")

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # calls Parent's __init__
        self.age = age
        print(f"Child initialized with age {self.age}")

c = Child("Alice", 12)
```

Output

```
Parent initialized with Alice
Child initialized with age 12
```

The `super()` keyword can also be used for multiple inheritance. Multiple inheritance can be defined as when multiple child classes use the same parent class.

Example

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        super().show()
        print("B")

class C(A):
    def show(self):
        super().show()
        print("C")

class D(B, C):
    def show(self):
        super().show()
        print("D")

d = D()
d.show()
```

Output

```
A
C
B
D
```

1.3 Pandas questions

1.3.1 Which metric is better to fill missing data for a column - mean or median?

Whether mean or median should be used to fill missing values depends on the distribution and nature of the data.

1. Mean imputation

- Replace missing values with the mean (average) of the column.
- Works well when the data is normally distributed (symmetric, no extreme outliers).
- **Code:** `data.fillna(data.mean(), inplace=True)`
- **Pros:**
 - Simple to calculate.
 - Preserves the overall mean of the dataset.

- **Cons:**
 - Sensitive to outliers. A single extreme value can skew the mean and give a misleading replacement.

2. Median imputation

- Replace missing values with the middle value of the sorted column.
- Works well for skewed distributions or when there are outliers.
- Code: `data.fillna(data.median(), inplace=True)`
- **Pros:**
 - Robust to outliers.
 - Maintains the central tendency better if data is skewed.
- **Cons:**
 - Does not preserve the mean.

1.3.2 What is the difference between `loc()` and `iloc()`?

1. `loc()`: label-based indexing

- Access rows/columns by labels (names).
- End of the range is inclusive.
- Can accept boolean conditions.
- `df.loc['b']` or `df.loc[['a', 'c'], ['Name']]`.
- Can use conditions, e.g., `df.loc[df['Age'] > 25]`.

2. `iloc()`: integer-based indexing

- Access rows/columns by integer positions (like Python lists).
- End of the range is exclusive.
- `df.iloc[1]` or `df.iloc[0:2, 0]`.
- Cannot use labels or boolean conditions directly.

1.3.3 What are the functions of `concat()`, `merge()` and `join()`?

In pandas, `concat()`, `merge()`, and `join()` are all used to combine DataFrames, but they work differently.

1. `concat()`: concatenation

- Used to stack DataFrames vertically (rows) or horizontally (columns).
- Does not require a common column.
- Mainly for simple combinations along an axis.
- `pd.concat([df1, df2], axis=0, ignore_index=False)`

2. `merge()`: database-style join

- Works like SQL JOIN.
- Requires a key column(s) to join on.
- Can perform inner, left, right, outer joins.
- `pd.merge(df1, df2, how='inner', on='key_column')`

3. `join()`: join on index

- Primarily used to join DataFrames on their index.
- Simpler alternative to `merge()` when working with indexes instead of key columns.
- Can also join on a column using `set_index()` first.
- `df1.join(df2, how='left')`

1.4 NumPy questions

1.4.1 What is vectorization and why is it faster?

In NumPy, vectorization refers to performing operations on whole arrays at once instead of using explicit Python loops over elements. This is a key feature that makes NumPy extremely fast.

Example (without vectorization)

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
result = np.zeros_like(arr)
```

```
for i in range(len(arr)):
    result[i] = arr[i] ** 2
```

```
print(result)
```

Example (with vectorization)

```
result = arr ** 2
print(result)
```

Why vectorization is faster

1. Underlying C/Fortran implementation

- NumPy operations are implemented in compiled C code, not Python loops.
- Python loops are interpreted, which is slower.

2. Optimized memory access

- Arrays are stored in contiguous blocks in memory.
- This allows faster access and computation using CPU-level optimizations.

3. Avoids Python overhead

- Each Python loop iteration involves function calls, type checking, and object creation.
- Vectorized operations work directly on the array's memory buffer.

4. SIMD and parallelism

- Modern CPUs can perform Single Instruction Multiple Data (SIMD) operations, processing multiple elements at once.
- NumPy takes advantage of this at a low level.

1.4.2 What is broadcasting?

In NumPy, broadcasting is a powerful mechanism that allows arrays of different shapes to be combined in arithmetic operations without explicitly reshaping them. It makes vectorized operations flexible and memory-efficient.

- When performing operations on arrays, NumPy automatically “stretches” the smaller array so that their shapes are compatible.
- No actual data is copied; NumPy just reuses the smaller array across the larger one.

Rules of broadcasting

1. Compare array shapes from the trailing dimensions.
2. Two dimensions are compatible if:
 - They are **equal**, or
 - One of them is **1**.
3. If shapes are not compatible, NumPy raises a `ValueError`.

Example 1: adding a scalar to an array

```
import numpy as np
arr = np.array([1, 2, 3])
result = arr + 5
print(result)
```

Output: [6 7 8]

Example 2: adding arrays of different shapes

```
a = np.array([[1,2,3],
               [4,5,6]]) # shape (2,3)
b = np.array([10,20,30]) # shape (3,)
result = a + b
print(result)
```

Output: [[11 22 33]
[14 25 36]]

Example 3: adding a column vector to a matrix

```
c = np.array([[1],[2]])      # shape (2,1)
d = np.array([[10,20,30]]) # shape (1,3)
result = c + d
print(result)
```

Output: `[[11 21 31]
 [12 22 32]]`

Why broadcasting is useful

- Enables vectorized operations without manual reshaping or loops.
- Saves memory, as arrays are not physically copied.
- Works for arithmetic, logical, and comparison operations.

1.5 Git questions

1.5.1 How does Git differentiate commits?

Git differentiates commits using a unique identifier called a SHA-1 hash.

1. Each commit is unique

- When a commit is created, Git generates a 40-character SHA-1 hash (a long hexadecimal string, e.g., a1b2c3d4...).
- This hash acts as the commit ID, uniquely identifying the commit in the repository.

2. Metadata of each commit

- Git calculates the hash based on:
 - The content of the files staged for commit.
 - Commit metadata:
 - Author name & email
 - Commit message
 - Timestamp
 - Parent commit(s) hash
- Even if only one character changes in a file or a different commit message is used, Git generates a completely different hash.

3. How Git tracks history

- Commits are linked to their parent commit via the parent hash.
- This forms a commit tree or DAG (Directed Acyclic Graph).
 - A <- B <- C
 - C points to B as parent, B points to A.

- Git uses hashes, not sequential numbers, so commit IDs are independent of order.

4. Other identifiers

- Branches and tags point to commit hashes, not the content itself.
- Git always uses the hash to check if two commits are the same or different, even across clones.

5. Why Git uses SHA-1

- SHA-1 produces a practically unique fingerprint of content.
- Guarantees integrity: changing a single character changes the hash drastically.

1.5.2 What are SHA keys?

SHA keys (or SHA hashes) are cryptographic identifiers used to uniquely represent data. In Git and many other applications, they are fundamental for integrity and security.

1. What is SHA?

- SHA stands for Secure Hash Algorithm.
- It's a hash function: it takes any input (file, string, etc.) and produces a fixed-length string of characters, usually hexadecimal.
- Even a tiny change in the input produces a completely different hash.
- **Common SHA types:**
 - SHA-1
 - SHA-256
 - SHA-512

2. SHA in Git

- Each commit, blob, tree, and tag in Git is identified by a SHA-1 hash.
- **Why Git uses SHA:**
 - **Uniqueness:** Each commit has a practically unique fingerprint.
 - **Integrity:** If files or metadata are altered, the SHA changes, so Git can detect tampering.
 - **History tracking:** Commits link to parent commits via SHA, forming a DAG (Directed Acyclic Graph).

3. Properties of SHA

- **Deterministic:** Same input gives you the same hash.
- **Fast to compute:** Even large files can be hashed quickly.
- **Irreversible:** You cannot reconstruct original data from SHA (one-way).
- **Avalanche effect:** Tiny change leads to a completely different hash.

1.5.3 What are the differences between fetching, pulling, cloning and forking?

In Git, `fetch`, `pull`, `clone`, and `fork` are all ways to get data from a repository, but they work differently.

1. `git clone`: copy a repository

- **Purpose:** Create a local copy of a remote repository for the first time.
- Downloads all files, branches, commits, and history.
- Creates a new directory with a `.git` folder.
- `git clone https://github.com/user/repo.git`

2. `git fetch`: download changes

- **Purpose:** Update your local copy of remote branches without merging them.
- Downloads new commits, branches, and tags from remote.
- **Does not** change your working files automatically.
- `git fetch origin`

3. `git pull`: fetch and merge

- **Purpose:** Update your local branch with remote changes and merge them immediately.
- `git pull = git fetch + git merge.`
- `git pull origin main`

4. Fork: copy a repository on GitHub/GitLab

- **Purpose:** Create your own remote copy of someone else's repository on GitHub/GitLab.
- Done on the server, not via Git commands.
- You can freely push changes to your fork without affecting the original repo.
- **Workflow Example:**
 - Fork repo on GitHub, this creates `your-username/repo`.
 - Clone your fork locally using `git clone`.
 - Make changes, commit, push to your fork.
 - Submit a pull request to merge into the original repo.

1.5.4 What is the function of `git log`? What are the other arguments one can add to it?

`git log` is used to view the commit history of a repository. It shows commits in reverse chronological order (most recent first) along with useful details like the commit hash, author, date, and message.

Common arguments of `git log`

Argument	Description
<code>--oneline</code>	shows each commit as a single line (short hash + message).
<code>--graph</code>	shows a text-based graph of branches and merges.
<code>--decorate</code>	shows branch names, tags, and refs along with commits.
<code>--author="name"</code>	filters commits by author.
<code>--since="yyyy-dd-mm"</code> <code>--until="yyyy-dd-mm"</code>	show commits after/before a certain date.
<code>-p</code> <code>--patch</code>	shows the diff introduced by each commit.
<code>--stat</code>	shows summary of changes (lines added/removed per file).
<code>-n <number></code>	limits output to last n commits.
<code>--reverse</code>	shows commits in chronological order (oldest first).
<code>--grep="keyword"</code>	filters commits whose messages match the keyword.
<code>--abbrev-commit</code>	shortens commit hashes (default in <code>--oneline</code>).

1.5.5 How are two branches merged?

Merging is the process of combining the changes from one branch into another.

Suppose we have two branches:

- `main` (the branch to merge into)
- `feature` (the branch with changes)

1. Basic merge

- Switch to target branch (`main`)
`git checkout main`
- Merge the other branch (`feature`) into it
`git merge feature`

2. Fast-forward merge

- If the target branch has no new commits since the branch diverged, Git just moves the pointer forward.
- No merge commit is created.

3. Three-way merge

- If both branches have new commits, Git performs a three-way merge:
 - Uses the common ancestor of the two branches.
 - Combines changes from both branches.
 - If conflicts exist, Git pauses for you to resolve them manually.

4. Handling conflicts

- If both branches change the same lines, a conflict occurs.
- Git marks conflicts in files like this:

```
<<<<<< HEAD
Changes in main branch
=====
Changes in feature branch
>>>>>> feature
```

- You manually edit the file to resolve the conflict, then:

```
git add <file>
git commit
```

5. Merge strategies

- **Recursive (default):** Handles two-parent merges (most common).
- **Fast-forward only:** Only allows fast-forward merges.
- **Squash merge:** Combines all feature branch commits into a single commit in main:

```
git merge --squash feature
```

1.5.6 Why can't merge commits be copied from two different branches?

1. What is a merge commit?

- A merge commit is a commit with two (or more) parent commits.
- It represents the point where two branches come together, preserving the history of both.

- Example:

```
A---B---C (main)
      \
        D---E (feature)
```

- If you merge `feature` into `main`, Git creates a new merge commit `M` with parents `C` and `E`.

```
A---B---C---M (main)
      \      /
        D---E (feature)
```

2. Why merge commits can't be copied

- **Parent commits are fixed**
 - Each commit in Git contains hashes of its parent(s).
 - A merge commit references exact commits from both branches.

- If you try to “copy” the merge commit to another branch where the parents don’t match, the commit hash would change, because the parents are different.
- **Merge commits depend on branch history**
 - The commit represents a merge of the specific histories of the two branches at that point in time.
 - Copying it elsewhere would break this integrity of history.
- **Git’s history is immutable**
 - Git hashes are content + metadata + parent hashes.
 - Changing parents (i.e., merging onto a different branch) changes the commit hash, so it is no longer the same commit.

3. How Git handles this

If you want to bring the same changes into another branch, Git offers:

- **Merge again:** creates a new merge commit with the correct parents.
- **Cherry-pick:** applies only the changes, creates a new commit on the target branch.
- **Example:**

```
# you have branch 'feature' merged into 'main'
# now you want it in 'develop' too
git checkout develop
git merge feature # creates a new merge commit for develop
```
- You cannot reuse the merge commit M from main, because its parents include commits from main, which don’t exist in develop’s history.

1.5.7 What are the different types of merges and how are they resolved?

1. Fast-forward merge

- **When it happens:** The target branch has no new commits since the feature branch diverged.
- **How it works:** Git simply moves the target branch pointer forward to the latest commit of the feature branch.
- No merge commit is created.

```
main:      A---B
feature:   C---D
```

```
git checkout main
git merge feature
```

Result:

```
main: A---B---C---D
```

Resolution: Automatic, no conflicts, no extra commit.

2. Three-way merge (recursive merge)

- **When it happens:** Both branches have new commits since they diverged.
- **How it works:** Git finds the common ancestor of the two branches and merges the changes.
- A merge commit is created to tie the histories together.

```
main:      A---B---C
feature:    B---D---E
```

```
git checkout main
git merge feature
```

```
Result:
A---B---C---M (main)
      \      /
      D---E (feature)
```

- M is the merge commit.

Resolution:

- If the changes affect different lines/files, Git merges automatically.
- If the changes conflict, Git marks the file for manual conflict resolution.

3. Squash merge

- **Purpose:** Combine all commits from a feature branch into a single commit on the target branch.

- **How it works:**

```
git merge --squash feature
git commit
```

Example: feature branch has 5 commits. After squash merge, 1 commit added to main.

- **Resolution:** Conflicts handled during squash; afterward, a single commit is created.

4. Octopus merge

- **Purpose:** Merge more than two branches at once.
- **Use case:** Usually for integration branches.

```
git merge branch1 branch2 branch3
```

Resolution:

- Works only if no complex conflicts exist across branches.
- If conflicts are complex, Git refuses and asks to merge branches individually.

5. **Rebase (alternative to merge)**

- Not a true merge, but often used to integrate changes linearly.
- **How it works:** Moves feature branch commits on top of the target branch.

```
git checkout feature  
git rebase main
```
- **Advantages:** Cleaner history, no merge commits.
- **Resolution:** Conflicts must be manually resolved during rebase.

6. **Conflict resolution**

Git marks conflicts like this:

```
<<<<<<< HEAD  
Changes in main branch  
=====  
Changes in feature branch  
>>>>>>> feature
```

Steps:

- Edit the file to resolve conflicts.
- Stage the file: `git add <file>`.
- Complete the merge: `git commit` (if not automatic).

1.5.8 **How do you use `git reset` and `git revert`? Which is used where?**

`git reset` and `git revert` are both used to undo changes, but they work differently and are used in different scenarios.

1. **`git reset`: move the HEAD**

- **Purpose:** Undo commits by moving the current branch pointer (HEAD) to a previous commit.
- Affects history, so can rewrite commit history. Use carefully on shared branches.

```
git reset [--soft | --mixed | --hard] <commit>
```
- **Types of reset:**

Type	Function	Use case
<code>--soft</code>	moves HEAD to <commit> but keeps changes staged	undo commit but keep staged for editing
<code>--mixed</code> (default)	moves HEAD and unstages changes (keeps in working dir)	undo commit, edit before re-committing
<code>--hard</code>	moves HEAD and discards all changes	permanently remove commits and changes

2. `git revert`: create an “undo” commit

- **Purpose:** Undo a commit by creating a new commit that reverses the changes.
- **Does not** rewrite history, safe for shared/public branches.

```
git revert <commit>
```

3. Which is used where

`git reset` is used for local undo of commits before pushing and for rewriting commit history.

`git revert` is used to undo commits safely on public/shared branches.

2. Machine learning

2.1 Machine learning algorithms

2.1.1 Supervised learning

Supervised learning is a type of machine learning where a model is trained on a labeled dataset — meaning the input data is paired with the correct output.

1. Training Data

- Each example has features (X) and a label/target (y).
- **Example:** Predicting house prices.
 - **Features:** size, number of rooms, location.
 - **Label:** actual price.

2. Learning Phase

- The algorithm learns a mapping function $f: X \rightarrow y$ that best predicts the output from the input.
- **Goal:** minimize the difference between predicted outputs and actual outputs (often using a loss function).

3. Prediction Phase

- Once trained, the model can take new, unseen inputs and predict their outputs.

Types of Supervised Learning

1. **Regression:** predicts continuous values.
Example: predicting temperature, stock prices.
2. **Classification:** predicts discrete categories.
Example: spam email detection (spam / not spam), disease diagnosis.

Examples of Algorithms

- Linear Regression
- Logistic Regression
- Decision Trees & Random Forests
- Support Vector Machines (SVM)
- k-Nearest Neighbors (kNN)
- Neural Networks

2.1.2 Unsupervised learning

Unsupervised learning is another type of machine learning, but unlike supervised learning, the data has no labels. The algorithm's job is to find hidden patterns, structures, or relationships in the input data without being told what the output should be.

Key idea

- **Data:** only features (X), no target labels (y).
- The model learns structure from the data itself.

Types of unsupervised learning

1. Clustering: grouping similar data points together

- **Example:** Market segmentation (grouping customers by buying habits), document grouping.
- **Algorithms:** k-Means, Hierarchical Clustering, DBSCAN.

2. Dimensionality reduction: simplifying data while retaining key information

- **Example:** Reducing 1000 features to 2 for visualization, compressing images.
- **Algorithms:** PCA (Principal Component Analysis), t-SNE, Autoencoders.

3. Association rule learning: finding relationships between variables

- **Example:** Market basket analysis ("People who buy bread often buy butter").
- **Algorithms:** Apriori, FP-Growth.

Examples in real life

- Recommender systems (e.g., Netflix suggesting shows you might like).
- Anomaly detection (e.g., spotting fraud in banking).
- Social network analysis (finding communities or influencers).

2.2 Linear regression

Linear regression is one of the simplest and most widely used algorithms in supervised learning. It is used to model the relationship between input features (X) and a continuous output variable (y). It assumes that the relationship between inputs and output is linear.

2.2.1 Univariate linear regression

It is a type of linear regression with only one independent variable (feature) and one dependent variable (target). It tries to fit a straight line through the data points that best describe the relationship.

Equation

$$f_{w,b}(x) = y = w + bx + \epsilon$$

- y = dependent variable (target)
- x = independent variable (input feature)
- w = intercept
- b = slope
- ϵ = error

Cost function

The cost function tells us how good or bad our line (or model) is at fitting the data. The most common cost function is the mean squared error (MSE).

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

- m = number of training samples
- $y^{(i)}$ = actual output for the i -th data point
- $\hat{y}^{(i)}$ = predicted output for the i -th data point
- J = cost (the function we want to minimize)

Gradient descent

It is an optimization algorithm used to minimize the cost function in linear regression, or for any other function.

The parameters w and b are updated by assigning:

$$w = w - \alpha \frac{\partial J}{\partial w}$$
$$b = b - \alpha \frac{\partial J}{\partial b}$$

- α = learning rate (step size)
- $\frac{\partial J}{\partial(w,b)}$ = gradient (slope of cost function)

Partial derivatives

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum (\hat{y} - y)x$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum (\hat{y} - y)$$

Implementing gradient descent

1. Initialize $\mathbf{w}$ and $\mathbf{b}$ randomly (e.g., 0).
2. Compute predictions $\hat{y}$.
3. Calculate the cost J .
4. Compute gradients (partial derivatives).
5. Simultaneously update parameters using gradient formulas.
6. Repeat until the cost stops decreasing (convergence).

Learning rate

The learning rate is one of the most important hyperparameters in gradient descent. It controls how big a step gradient descent takes when updating model parameters.

Effects of different learning rates

- If the learning rate is too small (e.g. 0.0001), the function converges very slowly and takes a lot of iterations.
- If the learning rate is too large (e.g. 1), the function may overshoot or oscillate but it never converges.
- If the learning rate is optimal (e.g. 0.01)

2.2.2 Multivariate linear regression

An extension of simple linear regression, but instead of one input variable, it uses two or more independent variables to predict the output.

$$f_{w,b}(x) = y = b + w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + \epsilon$$

- y = dependent variable (target)
- $x_1, x_2, x_3, \dots, x_n$ = independent variables (features)
- b = intercept
- $w_1, w_2, w_3, \dots, w_n$ = coefficients
- ϵ = error

The equation can also be represented vectorially.

$$f_{w,b}(\bar{x}) = \bar{w} \cdot \bar{x} + b$$

Where

$$\bar{w} = [w_1 \quad w_2 \quad w_3 \quad \dots \quad w_n]$$

$$\bar{x} = [x_1 \quad x_2 \quad x_3 \quad \dots \quad x_n]$$

Cost function

The parameters are updated to

$$w_1 = w_1 - \alpha \frac{\partial J}{\partial w_1}$$

$$w_2 = w_2 - \alpha \frac{\partial J}{\partial w_2}$$

$$\dots$$

$$w_n = w_n - \alpha \frac{\partial J}{\partial w_n}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

Partial derivatives

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \Sigma (\hat{y} - y) x_1$$

$$\frac{\partial J}{\partial w_2} = \frac{1}{m} \Sigma (\hat{y} - y) x_2$$

$$\dots$$

$$\frac{\partial J}{\partial w_n} = \frac{1}{m} \Sigma (\hat{y} - y) x_n$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \Sigma (\hat{y} - y)$$

Feature scaling

Feature scaling means adjusting the range of your input features so they're on a similar scale, typically around 0 or 1. Without scaling, features with large values dominate the learning process, making gradient descent very slow or unstable.

Types of normalization

1. Mean normalization

$$x_1 = \frac{x_1 - \mu_1}{x_{\max} - x_{\min}}, x_2 = \frac{x_2 - \mu_2}{x_{\max} - x_{\min}}, x_3 = \frac{x_3 - \mu_3}{x_{\max} - x_{\min}}, \dots, x_m = \frac{x_m - \mu_m}{x_{\max} - x_{\min}}$$

where μ is the mean.

2. Z-score normalization

$$x_1 = \frac{x_1 - \mu_1}{\sigma_1}, x_2 = \frac{x_2 - \mu_2}{\sigma_2}, x_3 = \frac{x_3 - \mu_3}{\sigma_3}, \dots, x_m = \frac{x_m - \mu_m}{\sigma_m}$$

where μ is the mean and σ is the standard deviation.

2.2.3 Polynomial regression

A type of linear regression where the relationship between the input variable (x) and the output variable (y) is modeled as an n th-degree polynomial. It is still linear in parameters, but nonlinear in input features.

$$f_{w,b}(x) = b + w_1x + w_2x^2 + w_3x^3 + \dots + w_nx^n$$

- $f_{w,b}(x)$ = predicted value
- $x, x^2, x^3, \dots, x^n$ = polynomial features
- $b, w_1, w_2, w_3, \dots, w_n$ = parameters

2.3 Logistic regression

2.3.1 Definition

Logistic regression is used for classification, not regression. It predicts the probabilities of classes.

$$f_{w,b}^-(x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

$f_{w,b}^-(x)$ is always between 0 and 1. It is the probability of y being equal to 1.

2.3.2 Decision boundary

A decision boundary is the line or curve that separates the regions where the classifier predicts different classes, typically 0 vs 1. In logistic regression, it's the set of points where the model predicts exactly 50% probability, i.e. $f_{w,b}^-(x) = 0.5$.

To find the decision boundary, first set $f_{w,b}^-(x) = 0.5$.

$$\frac{1}{1 + e^{-(w \cdot x + b)}} = 0.5$$

Thus, $e^{-(w \cdot x + b)} = 1$. This implies that $w \cdot x + b = 0$.

Thus, decision boundary is where $\bar{w} \cdot \bar{x} + b = 0$.

Nonlinear decision boundaries

If polynomial features are added then the equation becomes nonlinear. The boundaries are curved (circles, ellipses etc.) For example,

$$f_{w,b}^-(x) = \frac{1}{1 + e^{-(w_1 x_1^2 + w_2 x_2^2 + b)}}$$

Then the decision boundary is curved.

2.3.3 Cost function

$$L(f_{w,b}^-(\bar{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{w,b}^-(\bar{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{w,b}^-(\bar{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

This function is called a logarithmic loss function or a log loss function.

- If $y^{(i)} = 1$, $f_{w,b}^-(\bar{x}^{(i)})$ should be close to 1, resulting in a small cost.
- If $y^{(i)} = 0$, $f_{w,b}^-(\bar{x}^{(i)})$ should be close to 0, resulting in a small cost.
- If predictions are wrong (e.g. predicting 0.9 when $y = 0$), cost becomes very large.

$$J(\bar{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{w,b}^-(\bar{x}^{(i)}), y^{(i)})$$

This is the cost function for logistic regression.

Simplified cost function

We can simplify the loss function.

$$L(f_{w,b}^-(\bar{x}^{(i)}), y^{(i)}) = -y^{(i)} \log(f_{w,b}^-(\bar{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{w,b}^-(\bar{x}^{(i)}))$$

This is the overall log loss function for all training samples.

This simplified function can be used in the cost function.

$$J(\bar{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)})$$

Substituting the simplified loss function,

$$J(\bar{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\bar{w},b}(\bar{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\bar{w},b}(\bar{x}^{(i)})) \right]$$

This is the overall cost function for logistic regression.

Gradient descent

The parameters can be updated:

$$\begin{aligned} w_j &= w_j - \alpha \frac{\partial J}{\partial w_j} \\ b &= b - \alpha \frac{\partial J}{\partial b} \end{aligned}$$

Partial derivatives

$$\begin{aligned} \frac{\partial J}{\partial w_j} &= \frac{1}{m} \sum (\hat{y} - y) x_j \\ \frac{\partial J}{\partial b} &= \frac{1}{m} \sum (\hat{y} - y) \end{aligned}$$

2.3.4 Overfitting

When a model is too complex and learns not only the patterns but also the noise in the training data. It performs very well on training data, but poorly on new/test data.

How to reduce overfitting and improve generalization

1. Use more data

- More data helps the model see a wider variety of examples which makes it harder to memorize noise.
- This is especially effective for deep learning models.

2. Regularization

Regularization adds a penalty to large parameter weights to keep the model simpler.

- **L2 regularization**

$$J = L + \lambda \sum_{j=1}^n w_j^2$$

- Keeps weights small and smooths the decision boundary.
- **L1 regularization**

$$J = L + \lambda \sum_{j=1}^n |w_j|$$

- Can even force some weights to become zero.

2.4 Regularized linear regression

Regularizing the parameters,

$$w_j = w_j - \alpha \frac{\partial J}{\partial w_j} + \frac{\lambda}{m} w_j$$
$$b = b - \alpha \frac{\partial J}{\partial b}$$

b does not need to be regularized.

2.5 Regularized logistic regression

Regularizing the parameters,

$$w_j = w_j - \alpha \frac{\partial J}{\partial w_j} + \frac{\lambda}{m} w_j$$
$$b = b - \alpha \frac{\partial J}{\partial b}$$

b does not need to be regularized.